

Advanced RVC Inference

A state-of-the-art web UI crafted to streamline rapid and effortless RVC inference — featuring a model downloader, voice splitter, batch inference, training pipeline, real-time conversion, and a full CLI.

MIT License

43 Optimizers

4 Vocoders

30+ F0 Methods

Table of Contents

01

Getting Started

Installation, GPU support, and running the app

02

Features

Complete feature overview and capabilities

03

CLI Reference



Full command-line interface documentation

04

Vocoder Guide

4 vocoders — ratings, features, recommendations

05

Optimizer Guide

43 optimizers — ratings, features, recommendations

06

Configuration

Environment variables, file paths, model setup

07

Contributing

How to contribute, coding style, PR process

08

Credits & License

Project credits and MIT license



Getting Started

Installation

Clone the repository and install dependencies. Advanced RVC Inference requires Python 3.10 or later and works on Windows, Linux, and macOS.

```
git clone https://github.com/ArkanDash/Advanced-RVC-Inference.git
cd Advanced-RVC-Inference
pip install -r requirements.txt
```

BASH

Or install from PyPI directly:

```
pip install git+https://github.com/ArkanDash/Advanced-RVC-Inference.git
```

BASH

GPU Support (CUDA)

For NVIDIA GPU acceleration, install the CUDA-enabled ONNX runtime after the base package:

```
pip install git+https://github.com/ArkanDash/Advanced-RVC-Inference.git
pip install onnxruntime-gpu
```

BASH



ZLUDA (AMD GPU)

ZLUDA allows CUDA applications to run on AMD GPUs. Just install PyTorch with ZLUDA support — Advanced RVC will auto-detect and configure itself. No additional setup is required beyond following the standard ZLUDA installation guide for your specific AMD GPU model.

```
# Follow the ZLUDA installation guide for your AMD GPU
# Then install Advanced RVC normally — ZLUDA is auto-detected
pip install git+https://github.com/ArkanDash/Advanced-RVC-Inference.git
```

BASH

Running the Application

Launch the Gradio web UI using one of the following methods. The interface will be available at <http://localhost:7860> by default.

```
# Launch the web UI
rvc-gui

# Or via Python module
python -m arvc.app.gui

# With a public share link (for remote access)
python -m arvc.app.gui --share
```

BASH

CLI Usage

For headless operation, use the command-line interface. The CLI provides access to all features including voice conversion, audio separation, training, and more.

```
# Voice conversion
rvc-cli infer -m model.pth -i input.wav -o output.wav
```

BASH

```
# Audio separation
rvc-cli uvr -i song.mp3

# Show all commands
rvc-cli --help
```

Google Colab

Two Colab notebooks are available for cloud-based usage. The full Web UI notebook provides the complete Gradio interface, while the CLI-only notebook offers a lightweight headless mode for automated workflows.

NOTEBOOK	DESCRIPTION
Full Web UI	Complete Gradio interface with all features
CLI Only	Lightweight headless mode for automated workflows

Easy GUI

A simplified interface for quick workflows with minimal configuration. Provides quick voice conversion with minimal settings, one-click training for the full pipeline, and quick model download from URLs.

```
rvc-cli serve --easy true
```

BASH



Features

Advanced RVC Inference provides a comprehensive suite of tools for voice conversion, training, and audio processing. The project is stable and mature, with ongoing development focused on security patches, dependency updates, and occasional feature improvements.

Voice Inference

Single & batch conversion, TTS, pitch shifting, formant shifting, audio cleaning, Whisper transcription

Audio Separation

Vocal/instrumental isolation (MDX-Net, Roformer, BS-Roformer), karaoke, reverb removal, denoising

Real-Time Conversion

Live mic voice conversion with VAD and low-latency processing

Training Pipeline

End-to-end training from dataset creation to model export with overtraining detection

Easy GUI

Simplified one-click interface for quick conversion and training

CLI

Full command-line interface via `rvc-cli` with 13 subcommands

Auto Download



Automatically downloads pretrained models from HuggingFace

ZLUDA Support

Full AMD GPU support via ZLUDA auto-detection

30+ Fo Methods

rmvpe, crepe, fcpe, harvest, hybrid, and many more

Training Optimizations

Gradient accumulation, torch.compile(), 8-bit Adam, DDP tuning

Push to Hub

Upload trained models directly to HuggingFace Hub

Supported Vocoders

Advanced RVC Inference supports the same vocoders as [Vietnamese-RVC](#). When training without pitch guidance (`pitch_guidance=False`), a plain HiFi-GAN generator (no NSF) is used automatically regardless of the selected vocoder.

VOCODER	DESCRIPTION	PITCH REQUIRED
Default (HiFi-GAN NSF)	HiFi-GAN with Neural Sine Filter. Adds harmonic sine wave injection for improved pitch accuracy. Recommended for best compatibility.	Yes
BigVGAN	Snake activations with Anti-Aliasing (SnakeBeta + AMP blocks). State-of-the-art audio quality.	Yes
MRF-HiFi-GAN	HiFi-GAN with Multi-Receptive Field fusion. Richer feature extraction with MRF blocks.	Yes
RefineGAN		Yes ☐

VOCODER	DESCRIPTION	PITCH REQUIRED
	U-Net based vocoder with parallel residual blocks and anti-aliased resampling. High-fidelity spectral detail.	

CLI Reference

A comprehensive command-line interface for Advanced RVC Inference. The CLI provides access to all features including voice conversion, model training, audio separation, and more — all from the terminal.

infer — Voice Conversion

Convert voice in an audio file using an RVC model. This is the primary command for voice inference.

```
rvc-cli infer -m <model> -i <input> [options]
```

BASH

Required Arguments

-m, --model Path to RVC model file (.pth or .onnx)

-i, --input Path to input audio file



Optional Arguments

`-o, --output` Output file path (auto-generated if not specified)

`-p, --pitch` Pitch shift in semitones (default: 0)

`-f, --format` Output format — wav, mp3, flac, ogg (default: wav)

`--index` Path to .index file for better quality

`--f0_method` F0 extraction method (default: rmvpe)

`--filter_radius` Filter radius for F0 smoothing (default: 3)

`--index_rate` Index strength 0.0–1.0 (default: 0.5)

`--rms_mix_rate` RMS mix rate (default: 1.0)

`--protect` Protect consonants 0.0–1.0 (default: 0.33)

`--hop_length` Hop length for processing (default: 64)

`--embedder_model` Embedding model (default: hubert_base)

`--resample_sr` Resample rate (0 = original, default: 0)

`--split_audio` Split audio before processing

`--checkpointing` Enable memory checkpointing

`--f0_autotune` Enable F0 autotune

`--f0_autotune_strength` Autotune strength (default: 1.0)

`--formant_shifting` Enable formant shifting

`--formant_qfrecncy` Formant frequency coefficient (default: 0.8)

`--formant_timbre` Formant timbre coefficient (default: 0.8)

`--clean_audio` Apply audio cleaning



`--clean_strength` Cleaning strength (default: 0.7)

```
rvc-cli infer -m artist_model.pth -i speech.wav -o converted.wav -p 2 \  
--index_rate 0.75 --f0_method rmvpe --clean_audio
```

BASH

uvr — Audio Separation

Separate vocals from instrumentals using UVR5. Supports multiple separation models and post-processing options.

```
rvc-cli uvr -i <input> [options]
```

BASH

Required Arguments

`-i, --input` Path to input audio file

Optional Arguments

`-o, --output` Output directory (default: ./audios/uvr)

`-f, --format` Output format (default: wav)

`--model` Separation model (default: MDXNET_Main)

`--karaoke_model` Karaoke model (default: MDX-Version-1)

`--reverb_model` Reverb model (default: MDX-Reverb)

`--denoise_model` Denoise model (default: Normal)

`--sample_rate` Output sample rate (default: 44100)

`--shifts` Number of predictions (default: 2)

`--batch_size` Batch size (default: 1)

`--overlap` Overlap between segments (default: 0.25)

`--aggression` Extraction intensity (default: 5)

`--hop_length` Hop length (default: 1024)

`--window_size` Window size (default: 512)

`--enable_tta` Enable test-time augmentation

`--enable_denoise` Enable denoising

`--separate_backing` Separate backing vocals

`--separate_reverb` Separate reverb

Available Separation Models

MDXNET_Main, MDXNET_9482, HP-Vocal-1, HP-Vocal-2, Inst_HQ_1 through Inst_HQ_5, Kim_Vocal_1, Kim_Vocal_2

```
rvc-cli uvr -i song.wav --model HP-Vocal-2 --aggression 10 \  
  --enable_denoise --output ./vocals
```

BASH

create-dataset — Create Training Data

Create a training dataset from YouTube videos or local audio files. Automatically handles vocal separation and audio formatting.



BASH

```
rvc-cli create-dataset -u <url> [options]  
# or  
rvc-cli create-dataset -i <directory> [options]
```

Required Arguments (one of)

-u, --url YouTube URL (separate multiple with commas)

-i, --input Input directory with audio files

Optional Arguments

-o, --output Output directory (default: ./dataset)

--sample_rate Sample rate (default: 48000)

--clean_dataset Apply data cleaning

--clean_strength Cleaning strength (default: 0.7)

--separate Separate vocals (default: True)

--separator_model Separation model (default: MDXNET_Main)

--skip_start Seconds to skip at start (default: 0)

--skip_end Seconds to skip at end (default: 0)

BASH

```
rvc-cli create-dataset -u "https://youtube.com/watch?v=xxx" \  
  --sample_rate 48000 --separate --output ./my_dataset
```



create-index — Create Model Index

Create a .index file for voice retrieval. Indexing improves inference quality by enabling approximate nearest-neighbor search over training embeddings.

```
rvc-cli create-index <model_name> [options]
```

BASH

model_name Name of the model (positional)

--version RVC version — v1 or v2 (default: v2)

--algorithm Index algorithm — Auto, Faiss, or KMeans (default: Auto)

```
rvc-cli create-index mymodel --version v2 --algorithm Faiss
```

BASH

extract — Feature Extraction

Extract embeddings and F0 from training data. This step generates the feature files needed for model training.

```
rvc-cli extract <model_name> --sample_rate <rate> [options]
```

BASH

Required Arguments

model_name Name of the model (positional)

--sample_rate Sample rate of input audio



Optional Arguments

`--version` RVC version — v1 or v2 (default: v2)

`--f0_method` F0 extraction method (default: rmvpe)

`--f0_onnx` Use ONNX F0 predictor

`--pitch_guidance` Use pitch guidance (default: True)

`--hop_length` Hop length (default: 128)

`--cpu_cores` CPU cores (default: 2)

`--gpu` GPU index (default: - for CPU)

`--embedder_model` Embedder model (default: hubert_base)

`--rms_extract` Extract RMS energy

```
rvc-cli extract mymodel --sample_rate 48000 --f0_method rmvpe \  
  --gpu 0 --pitch_guidance
```

BASH

preprocess — Data Preprocessing

Slice and normalize training audio. This step prepares raw audio data for the feature extraction and training stages.

```
rvc-cli preprocess <model_name> --sample_rate <rate> [options]
```

BASH

Required Arguments

`model_name` Name of the model (positional)

`--sample_rate` Sample rate

Optional Arguments

`--dataset_path` Dataset path (default: ./dataset)

`--cpu_cores` CPU cores (default: 2)

`--cut_method` Cutting method — Automatic, Simple, or Skip (default: Automatic)

`--process_effects` Apply preprocessing effects

`--clean_dataset` Clean dataset

`--chunk_len` Chunk length for Simple method (default: 3.0)

`--overlap_len` Overlap length (default: 0.3)

`--normalization` Normalization mode — none, pre, or post (default: none)

```
rvc-cli preprocess mymodel --sample_rate 48000 --cut_method Automatic \  
--process_effects --normalization pre
```

BASH

train — Model Training

Train a new RVC voice model. Supports 43 optimizers and 4 vocoders for maximum flexibility and quality.

```
rvc-cli train <model_name> [options]
```

BASH



Required Arguments

`model_name` Name of the model (positional)

Optional Arguments

`--version` RVC version — v1 or v2 (default: v2)

`--author` Model author name

`--epochs` Total training epochs (default: 300)

`--batch_size` Batch size (default: 8)

`--save_every` Save checkpoint every N epochs (default: 50)

`--save_latest` Save only latest checkpoint (default: True)

`--save_weights` Save all model weights (default: True)

`--gpu` GPU index (default: 0)

`--cache_gpu` Cache data in GPU

`--pitch_guidance` Use pitch guidance (default: True)

`--pretrained_g` Path to pre-trained G weights

`--pretrained_d` Path to pre-trained D weights

`--vocoder` Vocoder — Default, BigVGAN, MRF-HiFi-GAN, RefineGAN (default: Default)

`--energy` Use RMS energy

`--overtrain_detect` Enable overtraining detection

`--optimizer`

Optimizer — 43 available (default: AdamW). Top rated: ScheduleFreeAdamW, Muon, S
Lion, Prodigy, NAdam



`--multiscale_loss` Use multi-scale mel loss

`--use_reference` Use custom reference set

`--reference_path` Path to reference set

```
rvc-cli train mymodel --version v2 --epochs 500 --batch_size 8 \  
  --gpu 0 --save_every 100 --vocoder "BigVGAN"
```

BASH

create-ref — Create Reference Set

Create reference audio for better inference quality. Reference sets help improve voice conversion accuracy.

```
rvc-cli create-ref <audio_file> [options]
```

BASH

Required Arguments

`audio_file` Path to audio file (positional)

Optional Arguments

`-n, --name` Reference name (default: reference)

`--version` RVC version — v1 or v2 (default: v2)

`--pitch_guidance` Use pitch guidance (default: True)

`--energy` Use RMS energy

`--embedder_model` Embedder model (default: hubert_base)

`--f0_method` F0 extraction method (default: rmvpe)

`--pitch_shift` Pitch shift (default: 0)

`--filter_radius` Filter radius (default: 3)

`--f0_autotune` Enable F0 autotune

`--alpha` Alpha blending (default: 0.5)

```
rvc-cli create-ref reference_audio.wav -n myref --f0_method rmvpe
```

BASH

download — Download Models/Audio

Download models from HuggingFace or audio from YouTube.

```
rvc-cli download -l <link> [options]
```

BASH

`-l, --link` Download link (HuggingFace or YouTube URL)

`-t, --type` Download type — model, audio, or index (default: model)

`-n, --name` Name to save as

```
rvc-cli download -l "https://huggingface.co/user/model/resolve/main/model.pth"
```

BASH



serve — Web Interface

Launch the Gradio web UI with configurable host and port settings.

```
rvc-cli serve [options]
```

BASH

--host Host to bind (default: 0.0.0.0)

--port Port to bind (default: 7860)

--share Create public share URL

```
rvc-cli serve --port 7860 --share
```

BASH

info — System Information

Display system and environment information including operating system and version, CPU information, memory and disk space, GPU information (if available), and Python/package versions.

```
rvc-cli info
```

BASH

version — Version Info

Show version and dependency information.

```
rvc-cli version
```

BASH



list-models — List Available Models

List installed models in the weights folder.

```
rvc-cli list-models
```

BASH

list-f0-methods — List Fo Methods

Show all available pitch extraction methods.

```
rvc-cli list-f0-methods
```

BASH

Complete Training Workflow

Follow these steps to train an RVC model from scratch using the CLI:

```
# 1. Create dataset from YouTube
```

```
rvc-cli create-dataset -u "https://youtube.com/watch?v=xxx" \  
    --output ./dataset --sample_rate 48000 --separate
```

```
# 2. Preprocess data
```

```
rvc-cli preprocess mymodel --sample_rate 48000 --cut_method Automatic
```

```
# 3. Extract features
```

```
rvc-cli extract mymodel --sample_rate 48000 --f0_method rmvpe --gpu 0
```

```
# 4. Train model
```

```
rvc-cli train mymodel --version v2 --epochs 300 --batch_size 8 --gpu 0
```

BASH

```
# 5. Create index for the model
rvc-cli create-index mymodel --version v2
```

Voice Conversion Examples

```
# Using RMVPE (recommended)
rvc-cli infer -m model.pth -i input.wav -o output_rmvpe.wav --f0_method rmvpe

# Using Harvest (faster)
rvc-cli infer -m model.pth -i input.wav -o output_harvest.wav --f0_method harvest

# Using Crepe (most accurate but slow)
rvc-cli infer -m model.pth -i input.wav -o output_crepe.wav --f0_method crepe-medium

# Batch processing
for file in ./inputs/*.wav; do
    rvc-cli infer -m model.pth -i "$file" -o "./outputs/${basename $file}"
done
```

Troubleshooting

"Model file not found"

Ensure the model path is correct. Check that the model file has `.pth` or `.onnx` extension and verify file permissions.

"CUDA out of memory"

Reduce batch size with `--batch_size 4`, enable checkpointing with `--checkpointing`, or use CPU with `--gpu -`.

"Audio format not supported"

Convert audio to WAV first using `ffmpeg -i input.mp3 output.wav`. Supported formats include wav, mp3, flac, ogg, opus, m4a, and aac.

"Fo method not available"

Install ONNX runtime for some methods. Some F0 methods require specific embedders to be available.

Vocoder Reference Guide

Advanced RVC Inference supports **4 vocoders** for audio synthesis, matching the vocoder support from [Vietnamese-RVC \(VRVC\)](#). Each vocoder has a different architecture, strengths, and quality characteristics. This guide provides detailed descriptions, ratings, and recommendations.

Quick Reference

RATING	VOCODER	CATEGORY	KEY FEATURE
★★★★★	Default (HiFi-GAN NSF) <code>default</code>	HiFi-GAN	Neural Sine Filter, harmonic injection
★★★★★	BigVGAN	Anti-Aliased GAN	SnakeBeta + AMP blocks, highest quality 

RATING	VOCODER	CATEGORY	KEY FEATURE
★★★★½	MRF-HiFi-GAN	Multi-Receptive Field	MRF blocks for richer features
★★★★	RefineGAN	U-Net GAN	Skip connections, parallel ResBlocks

Default (HiFi-GAN NSF) default

Rating: 5.0/5

Category: HiFi-GAN

Source: `models/generators/nsf_hifigan.py`

Registry Key: "Default"

The Default vocoder is the HiFi-GAN with Neural Sine Filter (NSF), and the recommended vocoder for best compatibility. It combines HiFi-GAN's transposed convolution upsampling with a Neural Sine Filter that injects harmonic information directly into each upsampling layer. The NSF source module generates sine waves conditioned on F0, which are mixed with the upsampled features through noise convolution layers. This vocoder provides improved pitch accuracy compared to standard HiFi-GAN due to the explicit harmonic conditioning. It is the default vocoder selected in both the UI and CLI, and the only vocoder available for V1 models.

Key Features:

- Neural Sine Filter for harmonic injection at each upsampling layer
- Noise convolutions for mixing harmonic and learned features
- Improved pitch accuracy from explicit harmonic conditioning
- Supports gradient checkpointing
- Requires pitch guidance (f0)
- Default selection in UI and CLI



Recommended for: Best compatibility. The default choice for all training. Works best when pitch accuracy is critical, such as singing voice and tonal languages.

BigVGAN

Rating: 5.0/5

Category: Anti-Aliased GAN

Source: `models/generators/bigvgan.py`

Registry Key: "BigVGAN"

BigVGAN is the highest-quality vocoder available in the system. It introduces two key innovations: Snake activations with Anti-Aliasing (SnakeBeta and Anti-Aliased Multi-Period/AMP blocks) and data-augmented adversarial training. The Snake activation function provides a periodic, non-monotonic nonlinearity that is naturally suited for audio signals, while the anti-aliased design prevents high-frequency artifacts during upsampling. BigVGAN uses kaiser-sinc filters for both upsampling and downsampling, achieving state-of-the-art audio quality across multiple benchmarks. Its architecture includes extensive AMP blocks with parallel branches at different periods, capturing both fine and coarse spectral details. During training, BigVGAN uses the v3 discriminator for improved adversarial signal.

Paper: "BigVGAN: A Universal Neural Vocoder with Large-Scale Training" (2023)

Key Features:

- SnakeBeta learnable periodic activations
- Anti-Aliased Multi-Period (AMP) blocks
- Kaiser-sinc filters for upsampling/downsampling
- Data-augmented adversarial training
- Superior high-frequency reconstruction
- Uses v3 discriminator during training



- Uses fp32 at inference (fp16 disabled for stability)

Recommended for: Maximum audio quality. Best for singing voice conversion and high-fidelity speech synthesis where quality is the top priority.

MRF-HiFi-GAN

Rating: 4.5/5

Category: Multi-Receptive Field

Source: `models/generators/mrf_hifigan.py`

Registry Key: "MRF-HiFi-GAN"

MRF-HiFi-GAN replaces the standard residual blocks with Multi-Receptive Field (MRF) blocks. Each MRF block contains a sequence of MRFLayers with different dilation stacks, allowing the network to capture features at multiple temporal scales simultaneously. This multi-scale approach is particularly effective for speech synthesis because speech contains information at multiple time scales — from fine-grained spectral details to broader prosodic patterns. The SineGenerator provides harmonic conditioning with `harmonic_num=8`. The synthesizer also accepts `"MRF HiFi-GAN"` (with space instead of hyphen) as an alias for backward compatibility.

Key Features:

- Multi-Receptive Field fusion blocks
- Multiple dilation stacks per block
- SineGenerator with 8 harmonics
- Richer multi-scale feature extraction
- Supports gradient checkpointing
- Uses fp32 at inference (fp16 disabled for stability)



Recommended for: Speech with complex spectral characteristics. Good for multi-speaker models where diverse voice qualities need to be captured across different temporal scales.

RefineGAN

Rating: 4.0/5

Category: U-Net GAN

Source: [models/generators/refinegan.py](#)

Registry Key: "RefineGAN"

RefineGAN uses a U-Net architecture with skip connections, a significant departure from the purely feedforward design of HiFi-GAN. The harmonic downsampling path processes F0 through sine generation, pre-convolution, and progressive downsampling using torchaudio's resample function. The upsampling path uses ParallelResBlocks with three parallel branches (kernel sizes 3, 7, 11) combined through AdaIN noise injection. Skip connections from the encoder to decoder preserve fine spectral details that might otherwise be lost during the compression-expansion process. During training, RefineGAN uses the v3 discriminator for improved adversarial signal.

Key Features:

- U-Net architecture with skip connections
- Parallel ResBlocks (3/7/11 kernel branches)
- AdaIN noise injection
- Anti-aliased harmonic downsampling
- Progressive refinement through skip connections
- Uses v3 discriminator during training
- Uses fp32 at inference (fp16 disabled for stability)



Recommended for: High-fidelity audio where spectral detail preservation is important. Good for singing and complex vocal passages where fine-grained detail matters.

Non-fo Mode (Plain HiFi-GAN)

When training **without pitch guidance** (`pitch_guidance=False`), the synthesizer automatically uses a plain HiFi-GAN generator (`HiFiGANGenerator` from `models/generators/hifigan.py`) regardless of the vocoder name selected. This is a separate, simpler HiFi-GAN without the Neural Sine Filter — it uses standard transposed convolution upsampling with weight-normalized residual blocks. The vocoder selection in the UI is locked to "Default" when pitch guidance is disabled.

UI Business Rules

The following rules are enforced by the UI (`arvc/ui/feedback.py`):

- **V1 models** can only use the **Default** vocoder (locked via `unlock_vocoder()`)
- **V2 models** can use all 4 vocoders
- **No pitch guidance (f0=False)** → vocoder is locked to **Default** (via `vocoders_lock()`)
- **Non-Default vocoders** force pitch guidance ON (via `pitch_guidance_lock()`)
- **Non-Default vocoders** use **fp32** at inference (fp16 disabled for stability)



Recommendations for RVC Training

Beginner

Use **Default (HiFi-GAN NSF)**. It's the default for a reason — best compatibility, good quality, and works reliably across all scenarios. The harmonic injection improves pitch accuracy out of the box.

Intermediate

Try **BigVGAN** for the highest audio quality. It consistently achieves the best objective and subjective quality scores across all benchmarks. The Snake activations and anti-aliased design produce noticeably cleaner output.

Advanced

Experiment with **MRF-HiFi-GAN** for multi-scale feature extraction, or **RefineGAN** for spectral detail preservation through its U-Net skip connections. Both offer unique quality characteristics for specific use cases.

Maximum Quality

Use **BigVGAN** — it consistently achieves the highest objective and subjective quality scores across all benchmarks.

Technical Notes

- V1 models are locked to the Default vocoder (enforced by UI)
- Non-Default vocoders require **v2 + pitch guidance** (enforced by UI)



- Non-Default vocoders use **fp32** at inference (fp16 disabled for stability)
- Vocoder choice is saved in the model checkpoint and used automatically during inference
- Pre-trained weights for non-Default vocoders follow the naming pattern:
`{VocoderName}_f0G48k.pth`
- BigVGAN and RefineGAN use the v3 discriminator during training
- The vocoder registry is in `arvc/engine/models/generators/__init__.py`

Optimizer Reference Guide

Advanced RVC Inference supports **43 optimizers** for model training, each with different characteristics, strengths, and use cases. This guide provides detailed descriptions, ratings, and recommendations for RVC/audio model training.

Quick Reference

RATING	OPTIMIZER	CATEGORY	BEST FOR
★★★★★	AdamW <code>default</code>	PyTorch Built-in	General-purpose, most reliable
★★★★★	ScheduleFreeAdamW	Schedule-Free	No LR schedule needed
★★★★★	Muon	Second-Order	Large models, convergence

RATING	OPTIMIZER	CATEGORY	BEST FOR
★★★★★	Sophia	Second-Order	Large-scale training
★★★★★½	Lion	Sign-Based	Memory-efficient training
★★★★★½	Prodigy	LR-Free	No LR tuning needed
★★★★★½	NAdam	PyTorch Built-in	Faster than standard Adam
★★★★★	RAdam	PyTorch Built-in	Warmup-free training
★★★★★	Adan	Nesterov	Vision and audio tasks
★★★★★	AnyPrecisionAdamW	Mixed-Precision	Bfloat16 training
★★★★★	Ranger21	Combined	RAdam + Lookahead synergy
★★★★★	AdaFactor	Memory-Efficient	Large model training
★★★★★	DAdaptAdam	LR-Free	Automatic LR from gradients
★★★★★	Adam	PyTorch Built-in	Classic adaptive optimizer
★★★★★	PAdam	Partial Adaptive	Adam-SGD interpolation
★★★★★	Apollo	Quasi-Newton	L-BFGS-like convergence
★★★★★½	CAME	Unified	Adam+SGD benefits combined

RATING	OPTIMIZER	CATEGORY	BEST FOR
★ ★ ★ ½	NovoGrad	Normalized	Well-conditioned gradients
★ ★ ★ ½	ScheduleFreeAdam	Schedule-Free	Adam without LR schedule
★ ★ ★ ½	DAdaptAdaGrad	LR-Free	Auto LR with AdaGrad
★ ★ ★	SGD	PyTorch Built-in	Best generalization
★ ★ ★	RMSprop	PyTorch Built-in	RL and recurrent networks
★ ★ ★	AdaBelief	Belief-Based	Better conditioned updates
★ ★ ★	AdaBeliefV2	Belief-Based	Stable deep training
★ ★ ★	LAMB	Layer-Adaptive	Large-batch training
★ ★ ★	LARS	Layer-Adaptive	Distributed training
★ ★ ½	Adagrad	PyTorch Built-in	Sparse data
★ ★ ½	Adadelata	PyTorch Built-in	No manual LR needed
★ ★ ½	Adamax	PyTorch Built-in	Robust to outliers
★ ★ ½	ASGD	PyTorch Built-in	Convex optimization
★ ★ ½	DAdaptSGD	LR-Free	SGD with auto LR
★ ★ ½	QHAdam	Quasi-Hyperbolic	Adam-SGD continuum
★ ★ ½	SWATS	Hybrid	Adam to SGD switching 

RATING	OPTIMIZER	CATEGORY	BEST FOR
★ ★ ½	Shampoo	Preconditioned	Layer preconditioning
★ ★ ½	SOAP	Second-Order	Distributed 2nd order
★ ★	A2Grad	Optimal Averaging	Theoretical guarantees
★ ★	AggMo	Aggregate Momentum	Multi-scale momentum
★ ★	PID	Control Theory	Novel control approach
★ ★	Yogi	Controlled Growth	Stable variance
★ ★	Fromage	Functional Regularization	Simple baseline
★ ★	SM3	Memory-Efficient	Sublinear memory
★ ★	ScheduleFreeSGD	Schedule-Free	SGD without schedule
★ ★	Nero	Normalized	Weight normalization

Tier 1: Best for RVC/Audio Training

AdamW

default

Rating: 5.0/5

Category: PyTorch Built-in

Source: `torch.optim.AdamW`

Adam with decoupled weight decay is the gold standard optimizer for deep learning training. It combines the adaptive learning rate of Adam with pr L2 regularization by decoupling weight decay from the gradient update. This

is the **default and recommended optimizer** for RVC model training. It provides reliable convergence across a wide range of model architectures, dataset sizes, and training configurations. The weight decay is applied directly to the weights rather than through the gradient, which leads to more consistent regularization behavior regardless of the learning rate.

Key Features: Adaptive learning rates per parameter, decoupled weight decay (proper L2 regularization), fused CUDA kernel support for faster training, proven track record across all of deep learning, well-understood behavior and debugging.

Recommended for: All RVC training scenarios as the default choice. Works well with learning rates between $1e-4$ and $1e-3$, batch sizes 4-32, and 100-1000 epochs.

ScheduleFreeAdamW

Rating: 5.0/5

Category: Schedule-Free

Schedule-Free AdamW eliminates the need for any learning rate scheduling by maintaining a dual set of parameters. The "z" parameters serve as a lookahead while "y" parameters follow standard AdamW updates. The optimizer dynamically adjusts its effective learning rate based on the distance between z and y, providing built-in warmup at the start of training and natural decay as convergence approaches. This means you never need to worry about warmup steps, cosine annealing, or step decay schedules again.

Key Features: No learning rate schedule needed whatsoever, built-in warmup phase (first ~5% of training), automatic decay as training converges, drop-in replacement for AdamW, stable across different model sizes.



Recommended for: Users who want to avoid learning rate schedule tuning. Especially useful when training with varying dataset sizes or when you're unsure what schedule to use.

Muon

Rating: 5.0/5

Category: Second-Order

Muon applies Newton-Schulz iteration to orthogonalize the momentum vector at each step. This normalization provides significantly better conditioning for the optimization landscape, similar in spirit to preconditioning in second-order methods but at a much lower computational cost. Muon has gained popularity for training large language models, where it demonstrates faster convergence compared to AdamW, particularly in later training stages. The orthogonalization ensures that updates move in well-conditioned directions, reducing the chance of oscillation or stagnation.

Key Features: Momentum orthogonalization via Newton-Schulz iteration, better conditioned optimization landscape, faster convergence on deep models, popularized for large-scale language model training, works well with high learning rates.

Recommended for: Advanced users training large RVC models (v2, 48k) who want faster convergence. Particularly effective with 300+ epoch training runs.

Sophia

Rating: 5.0/5

Category: Second-Order



Sophia is a second-order optimizer that uses a diagonal Hessian estimate combined with a stochastic clipping mechanism. Unlike Adam which only uses first-order gradient information, Sophia incorporates curvature information from the Hessian (second derivatives) to make more informed update decisions. The diagonal approximation keeps memory usage manageable while still providing significant convergence benefits. The clipping mechanism prevents excessively large updates in high-curvature directions, ensuring training stability.

Key Features: Diagonal Hessian estimation for curvature awareness, stochastic clipping for stability, faster convergence than first-order methods, memory-efficient diagonal approximation, update frequency control via k parameter.

Recommended for: Users with sufficient GPU memory who want maximum convergence speed. Best with larger batch sizes (8+) and longer training runs.

Tier 2: Excellent Optimizers

Lion

Rating: 4.5/5

Category: Sign-Based

Lion (EvoLved Sign Momentum) was discovered through automated program search rather than manual design. Its key innovation is using the sign of the momentum rather than the momentum itself for the update direction. This dramatically simplifies the computation: instead of dividing by the square root of the variance, Lion just takes the sign. This results in significantly lower memory usage (only one state tensor vs. two in Adam) and often



matches or exceeds AdamW's performance, particularly with higher learning rates.

Recommended for: Memory-constrained training scenarios or when you want to try a higher learning rate than AdamW allows without diverging.

Prodigy

Rating: 4.5/5

Category: LR-Free

Prodigy automatically determines the optimal learning rate by estimating the distance to the solution (D0) using gradient statistics. You only need to set one intuitive parameter: `d_coef` (what fraction of D0 to traverse per epoch). The optimizer continuously adapts its effective learning rate during training based on the ratio of parameter change to gradient magnitude. This eliminates the most common failure mode in training — choosing the wrong learning rate — while still allowing the optimizer to benefit from Adam's adaptive per-parameter updates.

Recommended for: Users who struggle with learning rate tuning or are training multiple models with different architectures and need a "set it and forget it" optimizer.

NAdam

Rating: 4.5/5

Category: PyTorch Built-in

NAdam combines Adam's adaptive learning rates with Nesterov accelerated gradient. The Nesterov aspect means the optimizer looks ahead by computing the gradient at the anticipated next position rather than the current position. This lookahead provides a form of implicit momentum

correction that often leads to faster convergence, especially in the early stages of training. NAdam is particularly well-suited for RVC training because audio model loss landscapes tend to benefit from the accelerated convergence that Nesterov momentum provides.

Recommended for: Users who want a slight upgrade over AdamW without the complexity of newer optimizers. Good default alternative to AdamW.

Tier 3: Very Good Optimizers

RAdam

Rating: 4.0/5

PyTorch Built-in

Rectified Adam addresses a fundamental issue with Adam: during the first few training steps, the variance estimate is unreliable because it's computed from very few samples. RAdam dynamically rectifies this by switching between SGD-like updates (when variance is unreliable) and Adam-like updates (when variance becomes trustworthy). This eliminates the need for warmup steps that Adam typically requires.

Recommended for: Short training runs where warmup would consume a significant fraction of total steps.

Adan

Rating: 4.0/5

Nesterov

Adan introduces a unique third moment that tracks the difference between consecutive gradients. This gradient difference captures information about

the curvature of the loss landscape, effectively providing second-order information at first-order cost. The Nesterov-style momentum estimation further enhances convergence speed. Adan has shown particularly strong results on vision and audio tasks.

Recommended for: Audio/vision training tasks where gradient smoothness matters.

AnyPrecisionAdamW

Rating: 4.0/5

Mixed-Precision

AnyPrecisionAdamW is an AdamW variant with configurable data types for its internal momentum and variance buffers. This allows fine-grained control over numerical precision during mixed-precision training. When using bfloat16, this optimizer can maintain its statistics in bfloat16 or optionally use Kahan summation for enhanced numerical accuracy.

Recommended for: Users training with bfloat16 who want maximum numerical stability, especially for very long training runs (500+ epochs).

Ranger21

Rating: 4.0/5

Combined

Ranger21 synergistically combines RAdam's variance rectification with Lookahead's slow-fast weight synchronization. Every k steps, the optimizer interpolates between the current "fast" weights (updated by RAdam) and "slow" weights (updated less frequently). This periodic synchronization acts as a regularizer that prevents the optimizer from overshooting minima.

Recommended for: Users who want a "best of both worlds" optimizer with RAdam's stability and Lookahead's generalization benefits.

AdaFactor

Rating: 4.0/5

Memory-Efficient

AdaFactor dramatically reduces memory usage by factoring the second-moment estimator into row-wise and column-wise statistics instead of storing the full per-element variance tensor. For a parameter matrix of shape (m, n) , Adam stores $m \times n$ variance values while AdaFactor only stores $m + n$ values. It also uses a relative step size based on the RMS of the parameters themselves.

Recommended for: Training large RVC models on GPUs with limited memory.

DAdaptAdam

Rating: 4.0/5

LR-Free

DAdaptAdam automatically determines the learning rate by estimating the distance to the optimal solution from accumulated gradient statistics. The key insight is that the sum of squared gradients provides information about this distance. Set `lr=1.0` and let D-Adapt handle the rest.

Recommended for: Users who want automatic learning rate tuning while keeping the familiar Adam behavior.



Adam

Rating: 4.0/5

PyTorch Built-in

The original Adam optimizer remains one of the most widely used optimizers in deep learning. It combines first moment (mean) and second moment (uncentered variance) estimates with bias correction to provide per-parameter adaptive learning rates. While AdamW has largely replaced it due to better weight decay handling, Adam still performs well in many scenarios.

Recommended for: Users who want the classic Adam experience, or when comparing against existing results that used Adam.

PAdam

Rating: 4.0/5

Partial Adaptive

PAdam introduces a `p_partial` parameter that controls how much of the second moment's power to use. When `p_partial=0`, PAdam behaves like SGD; when `p_partial=1`, it behaves like Adam. The default `p_partial=0.25` provides a balance that retains some of Adam's adaptivity while gaining some of SGD's generalization benefits.

Recommended for: Users who want a balance between Adam's fast convergence and SGD's good generalization.

Apollo

Rating: 4.0/5

Quasi-Newton



Apollo approximates diagonal Hessian information using the ratio of consecutive gradients, similar to how L-BFGS builds up curvature information over time. This quasi-Newton approach provides second-order convergence benefits without the computational cost of full Hessian computation. The optimizer starts with Adam-like behavior and progressively incorporates more curvature information as training proceeds.

Recommended for: Users who want quasi-Newton convergence speed without the complexity and memory cost of full second-order methods.

Tier 4: Good Optimizers (3.5/5)

CAME — Closes the gap between Adam-style and SGD-style optimizers by tracking both the magnitude and sign consistency of gradients. Computes a "sign scale" that upweights updates when the gradient direction is consistent across steps.

NovoGrad — Normalizes the gradient by its RMS before computing the second moment, providing better conditioning across layers and more stable, predictable behavior.

ScheduleFreeAdam — Schedule-Free variant of standard Adam (without decoupled weight decay). Provides built-in warmup and decay for Adam without requiring external LR scheduling.

DAdaptAdaGrad — Combines AdaGrad's cumulative second moment with D-Adaptation's automatic learning rate estimation. Good performance on sparse or noisy gradient landscapes.



Tier 5: Solid Optimizers (3.0/5)

SGD — The foundational stochastic gradient descent optimizer. While simple, SGD with momentum and proper learning rate scheduling often provides the best generalization, especially on smaller datasets.

RMSprop — Maintains a moving average of squared gradients. Popular in reinforcement learning and recurrent network training where non-stationary gradient statistics benefit from decayed averaging.

AdaBelief — Adjusts step size based on the "belief" in the current gradient direction, computed as the difference between the current gradient and the exponential moving average of past gradients.

AdaBeliefV2 — Improved version of AdaBelief with AMSGrad support and better bias correction. The AMSGrad variant maintains the maximum of the variance estimates to prevent the learning rate from increasing.

LAMB — Layer-wise Adaptive Moments optimizer that applies a per-layer trust ratio to Adam updates. Essential for large-batch distributed training (BERT pre-training at scale).

LARS — Layer-wise Adaptive Rate Scaling computes a local learning rate for each layer based on the ratio of the layer's weight norm to its gradient norm, preventing any single layer from dominating the update.

Tier 6: Moderate Optimizers (2.5/5)

Adagrad — Accumulates the sum of squared gradients over all training steps. The learning rate for each parameter decreases as its accumulate

gradient grows, but the monotonic decrease can cause the learning rate to become too small.

Adadelta — Addresses Adagrad's monotonically decreasing learning rate by restricting the accumulation window to a fixed number of recent gradients.

Adamax — Adam variant that uses the infinity norm (maximum absolute value) instead of the L2 norm for the second moment, making it more robust to outliers in the gradient data.

ASGD — Averaged Stochastic Gradient Descent maintains a running average of all past parameter vectors. The final averaged parameters often generalize better than the last iterate.

DAdaptSGD — SGD with momentum combined with D-Adaptation's automatic learning rate. Provides SGD's generalization benefits without manual LR tuning.

QHAdam — Quasi-Hyperbolic Adam generalizes Adam via two discounting parameters (ν_1, ν_2) that control the interpolation between SGD and Adam.

SWATS — Starts training with Adam for fast initial convergence, then switches to SGD when the adaptive learning rate's variance drops below a threshold.

Shampoo — Uses layer-wise preconditioning by approximating the Hessian with Kronecker products of smaller matrices for better conditioning.

SOAP — Second-Order Adam-like Preconditioner uses distributed second-order information for better conditioned updates in large-scale distributed training.



Tier 7: Specialized/Niche Optimizers (2.0/5)

A2Grad — Stochastic Gradient Descent with optimal averaging of iterates. Uses second-order information to compute theoretically optimal step sizes.

AggMo — Aggregate Momentum maintains multiple momentum buffers simultaneously at different decay rates, combining fast adaptation with long-term memory.

PID — Applies Proportional-Integral-Derivative control theory concepts to gradient descent.

Yogi — Controls the growth rate of the second moment estimate to prevent the effective learning rate from increasing uncontrollably.

Fromage — Normalizes each parameter update by the Frobenius norm of its gradient and clamps it by the parameter norm.

SM3 — Squared Method of Moments maintains element-wise maximum of squared gradients for memory-efficient adaptation.

ScheduleFreeSGD — Schedule-Free variant of SGD with momentum, providing built-in warmup and decay.

Nero — Normalizes weight matrices at each step, providing built-in weight normalization that acts as a natural regularizer.



Recommendations for RVC Training

Beginner

Start with **AdamW** (default). It's the most tested and reliable optimizer for RVC training. Use learning rate $1e-3$ with 300 epochs and batch size 8.

Intermediate

Try **ScheduleFreeAdamW** to eliminate LR schedule tuning, or **NAdam** for slightly faster convergence. These are drop-in replacements that require no additional configuration.

Advanced

Experiment with **Sophia** or **Muon** for faster convergence on larger models. **Prodigy** and **DAdaptAdam** are excellent choices if you want to eliminate learning rate tuning entirely.

Memory-Constrained

Use **Lion** (50% less memory than Adam) or **AdaFactor** (sublinear memory scaling). Both provide good performance while reducing memory footprint.

Large-Batch Training

Use **LAMB** or **LARS** for their per-layer adaptive learning rate scaling, which prevents gradient explosion in large-batch scenarios.



Technical Notes

- All custom optimizers are implemented in `arvc/models/optimizers/`
- The central registry in `__init__.py` maps optimizer names to their classes
- The training engine (`engine/training/runner/train.py`) uses the registry for dynamic optimizer selection
- Each optimizer automatically receives appropriate kwargs (betas, eps, weight_decay) based on its capabilities
- Fused CUDA kernels are automatically enabled when supported (currently only AdamW)
- For optimizers that don't support `betas` or `eps` , these parameters are silently omitted

Configuration

Environment Variables

Advanced RVC Inference uses environment variables to customize paths for assets, configs, weights, and logs. These can be set before launching the application to override the default locations.

VARIABLE	DESCRIPTION	DEFAULT
ARVC_ASSETS_PATH	Path to assets directory	<code>assets</code>

VARIABLE	DESCRIPTION	DEFAULT
ARVC_CONFIGS_PATH	Path to configs directory	configs
ARVC_WEIGHTS_PATH	Path to weights directory	assets/weights
ARVC_LOGS_PATH	Path to logs directory	assets/logs

Model and Index Files

Place your model files (`.pth` or `.onnx`) in the weights directory. Place index files (`.index`) in the logs directory under the model name subfolder.

```
# Model files
arvc/assets/weights/

# Index files
arvc/assets/logs/<model_name>/
```

BASH

Terms of Use

The use of the converted voice for the following purposes is **strictly prohibited**:

- Criticizing or attacking individuals
- Advocating for or opposing specific political positions, religions, or ideologies
- Publicly displaying strongly stimulating expressions without proper zoning
- Selling of voice models and generated voice clips
- Impersonation of the original owner of the voice with malicious intentions
- Fraudulent purposes that lead to identity theft or fraudulent phone calls



Contributing

Whether you're fixing a typo, adding a feature, or reporting a bug — every contribution matters. This guide will help you get started without a ton of overhead.

Quick Start

BASH

```
# 1. Fork and clone
git clone https://github.com/YOUR-USERNAME/Advanced-RVC-Inference.git
cd Advanced-RVC-Inference

# 2. Set up upstream
git remote add upstream https://github.com/ArkanDash/Advanced-RVC-Inference.git

# 3. Install dependencies
pip install -e .

# 4. Create a branch, make changes, push, and open a PR!
```

Project Structure

TEXT

```
arvc/
├─ app/           # Gradio web UI (tabs, pages, layouts)
│  └─ tabs/       # inference, training, downloads, realtime, etc.
│     └─ easy_gui.py # simplified one-click interface
```



```

├─ engine/           # Core logic (no UI dependency)
|   ├─ inference/    #   voice conversion pipeline, TTS
|   ├─ training/     #   preprocess, extract, train, export
|   ├─ uvr/          #   audio separation (UVR5)
|   ├─ realtime/     #   live mic conversion
|   └─ models/       #   model loading, backends (CUDA, DirectML, OpenCL)
├─ services/        # Business logic layer (bridges UI ↔ engine)
├─ ui/              # UI helpers (feedback, dropdown updates, formatting)
├─ utils/           # Shared utilities (variables, download helpers)
├─ configs/         # Configuration files (config.json, training configs)
└─ assets/          # Runtime assets (models, languages, presets, weights)
    └─ languages/    #   44 translation JSON files

```

Key rule: `engine/` should never import from `app/` or `services/`. Keep the core independent.

Ways to Contribute

Reporting Bugs

Open an [issue](#) with: what you expected vs. what happened, steps to reproduce, error messages or logs, and your environment (OS, Python version, GPU, how you launched).

Writing Code

AREA	WHAT
UI/UX	Gradio interface improvements, new tabs, better layout
Translations	Fix or improve any of the 44 language files

AREA	WHAT
Core Engine	Inference optimizations, new F0 methods, training pipeline
Bug Fixes	Pick an open issue and go for it
Documentation	Tutorials, code comments, README improvements
Testing	Unit tests, integration tests — currently very limited

Coding Style

- **PEP 8** — standard Python style, 4 spaces, no tabs
- **Line length** — try to stay under 120 characters
- **Type hints** — appreciated for public functions, not required everywhere
- **Docstrings** — add them for new public functions and classes
- **Import order** — stdlib, then third-party, then local
- Use `translations.get("key", "fallback")` instead of `translations["key"]` — this prevents crashes when a translation key is missing
- Keep `engine/` free of UI imports — it should work headless
- Log errors with `logger.error()` and show user-facing messages with `gr_warning()` / `gr_error()` / `gr_info()`

Submitting Changes

When you're ready to submit your work, sync with upstream, push to your fork, and open a PR against the `master` branch. In your PR description, include what it does, why it's needed, how you tested it, and any related issues.

PR Checklist

- Does the code run without errors?



- Did you test the feature/fix manually?
- Are translation keys added to all language files (if you added new UI text)?
- No hardcoded strings that should be translatable?
- Event handler outputs match function return values?

Community

- **Discord:** <https://discord.gg/hvmsukmBHE>
- **GitHub Issues:** <https://github.com/ArkanDash/Advanced-RVC-Inference/issues>
- **GitHub Discussions:** <https://github.com/ArkanDash/Advanced-RVC-Inference/discussions>

Credits & License

Credits

PROJECT	AUTHOR	PURPOSE
Vietnamese-RVC	Pham Huynh Anh	Core RVC implementation & pretrained models
Applio	IAHispano	UI/UX inspiration & components
Mangio-Kalo-Tweaks	kalomaze	EasyGUI inspiration

PROJECT	AUTHOR	PURPOSE
python-audio-separator	Nomad Karaoke	UVR5 audio separation
whisper	OpenAI	Speech-to-text transcription
BigVGAN	Nvidia	Vocoder implementation
ZLUDA	vlsid	AMD GPU CUDA compatibility layer

License

This project is licensed under the **MIT License**. Copyright 2023 ArkanDash. See the [LICENSE](#) file for the full license text.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING



FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR
OTHER DEALINGS IN THE SOFTWARE.

Advanced RVC Inference v2.1.0 – Documentation compiled 2025

[GitHub](#) · [Discord](#)

